

# Literature Review: TinyML-Based Smart Intrusion Detection System with ESP32

## 1. Introduction

The proliferation of Internet of Things (IoT) devices has enabled the development of affordable and intelligent security solutions for residential and commercial environments. The EagleEye Smart Intrusion System represents a convergence of several cutting-edge technologies: Tiny Machine Learning (TinyML), edge computing, embedded vision systems, and real-time communication protocols<sup>[1]</sup><sup>[2]</sup>. This literature review examines the foundational research, current methodologies, and technological frameworks that underpin the development of a door monitoring system utilizing ESP32 microcontroller for local image processing with WhatsApp notification capabilities.

The core objective of this project is to implement person detection algorithms directly on resource-constrained embedded devices, specifically the ESP32-CAM module, which can capture images of individuals at doorways and transmit alerts via messaging platforms without relying on cloud-based processing<sup>[2]</sup><sup>[3]</sup>. This approach addresses critical concerns regarding privacy, latency, cost-effectiveness, and energy efficiency that characterize traditional cloud-dependent surveillance systems.

## 2. Tiny Machine Learning (TinyML) Fundamentals

### 2.1 Definition and Scope

Tiny Machine Learning (TinyML) represents the intersection of machine learning, embedded systems, and IoT technology, enabling the deployment of neural network models on microcontroller units (MCUs) with severely constrained computational resources<sup>[4]</sup><sup>[5]</sup>. TinyML systems typically operate within memory constraints of a few hundred kilobytes and power budgets measured in milliwatts, making them suitable for battery-powered, always-on applications<sup>[6]</sup><sup>[7]</sup>.

The TinyML paradigm has experienced remarkable growth since its formal introduction in 2019, with research publications exhibiting a "hockey stick effect" indicating rapid acceleration in both academic research and commercial applications<sup>[8]</sup>. This growth is driven by advances in model compression techniques, specialized hardware accelerators, and optimized inference frameworks designed specifically for microcontrollers.

### 2.2 Key Enabling Technologies

Several technological advances have made TinyML feasible for computer vision applications:

**Model Compression and Quantization:** Neural networks trained on powerful hardware must undergo substantial optimization before deployment on microcontrollers. Quantization techniques convert 32-bit floating-point weights to 8-bit integers (int8), reducing model size by approximately 75% while maintaining acceptable accuracy levels<sup>[9]</sup><sup>[10]</sup>. Research demonstrates that properly quantized models can achieve 97% accuracy retention compared to their full-precision counterparts<sup>[7]</sup>.

**TensorFlow Lite for Microcontrollers:** TensorFlow Lite Micro (TFLite Micro) has emerged as the de facto framework for deploying machine learning models on embedded devices<sup>[11]</sup>. The framework has been successfully ported to multiple architectures including ARM Cortex-M, ESP32 (Xtensa), and various digital signal processors (DSPs). TFLite Micro operates without requiring an operating system or standard C/C++ libraries, making it highly portable across diverse hardware platforms<sup>[11]</sup>.

**Edge Impulse Platform:** Edge Impulse has become a leading development platform for TinyML applications, providing end-to-end workflows for data collection, model training, and deployment to embedded devices<sup>[12]</sup><sup>[13]</sup><sup>[14]</sup>. The platform supports multiple target devices including ESP32 and Arduino boards, and offers the EON Compiler which optimizes TensorFlow Lite models for specific microcontroller architectures<sup>[14]</sup>.

### 2.3 Energy Efficiency Considerations

Energy efficiency represents a critical design constraint for battery-powered embedded vision systems. Research by Banbury et al. demonstrates that TinyML systems can achieve energy savings of up to four orders of magnitude compared to traditional machine learning systems<sup>[6]</sup>. For instance, microcontrollers like the Cortex-M7 consume approximately 23.5 mW during active inference, while the specialized NDP100 neural processor consumes only 150  $\mu$ W<sup>[6]</sup>.

A comprehensive study on TinyML energy viability for computer vision applications using ESP32 revealed that camera-equipped prototypes consumed 1.25 volts per hour, while configurations without camera sensors showed improved longevity at 0.93 volts per hour<sup>[6]</sup>. These findings emphasize the importance of intelligent power management strategies, including deep sleep modes and event-driven activation, to maximize operational lifetime in deployment scenarios.

### 3. ESP32 Platform for Embedded Vision

#### 3.1 Hardware Architecture

The ESP32 family of microcontrollers has emerged as a popular platform for TinyML applications due to its combination of processing capability, integrated wireless connectivity, and affordability<sup>[4]</sup>. The ESP32 features a dual-core 32-bit processor operating at up to 240 MHz, 520 KB of SRAM, and external flash memory support up to 16 MB<sup>[4]</sup> <sup>[15]</sup>. The dual-core architecture enables parallel processing, with one core dedicated to network communication while the other handles inference tasks.

The ESP32-CAM variant integrates an OV2640 camera module capable of capturing 2-megapixel images, making it particularly suitable for vision-based applications<sup>[2]</sup> <sup>[16]</sup>. The camera supports multiple resolution modes including UXGA (1600×1200), SVGA (800×600), and lower resolutions optimized for machine learning inference such as 96×96 pixels<sup>[2]</sup> <sup>[17]</sup>.

#### 3.2 Computational Capabilities and Limitations

While the ESP32 provides impressive capabilities for a microcontroller, it faces significant constraints when executing complex computer vision tasks. Community discussions reveal that the ESP32-CAM lacks sufficient processing power to handle advanced machine learning operations independently, particularly for real-time pattern recognition and anomaly detection<sup>[18]</sup> <sup>[19]</sup>. Developers frequently report challenges including:

- **Processing Bottlenecks:** Complex object detection models can overwhelm the ESP32's computational capacity, resulting in low frame rates (1-3 FPS) for inference<sup>[20]</sup> <sup>[21]</sup>
- **Memory Constraints:** Limited SRAM restricts the size and complexity of deployable neural network models<sup>[15]</sup>
- **Camera Initialization Issues:** Common technical challenges include "camera init failed" errors, brownout detection problems, and frame buffer timeout issues<sup>[22]</sup>

Despite these limitations, research demonstrates that carefully optimized TinyML models can achieve acceptable performance on ESP32 hardware. A study on camera image processing with convolutional neural networks proved that ESP32 capacity is sufficient for simultaneous operation of both camera acquisition and neural network inference when models are appropriately sized<sup>[15]</sup>.

#### 3.3 Successful Implementation Strategies

Research identifies several strategies for successful ESP32-based vision applications:

**Resolution Optimization:** Reducing input image resolution to 96×96 or 128×128 pixels significantly decreases computational requirements while maintaining sufficient information for person detection tasks<sup>[2]</sup> <sup>[17]</sup>. The Visual Wake Words dataset, widely used for TinyML person detection, employs 96×96 pixel grayscale images as the standard input format<sup>[17]</sup>.

**Model Architecture Selection:** Lightweight architectures such as MobileNet-SSD and FOMO (Faster Objects, More Objects) demonstrate optimal performance on resource-constrained devices<sup>[2]</sup> <sup>[23]</sup> <sup>[24]</sup>. MobileNet employs depthwise separable convolutions that reduce computational complexity while maintaining feature extraction capability<sup>[25]</sup>.

**Hybrid Edge-Cloud Approaches:** While the EagleEye project emphasizes local processing, some implementations employ hybrid architectures where the ESP32 performs preliminary detection and the cloud handles secondary verification, balancing latency with accuracy requirements<sup>[2]</sup>.

### 4. Person Detection Algorithms for Embedded Systems

#### 4.1 Dataset Requirements and Availability

High-quality datasets are fundamental to training effective person detection models for TinyML applications. The Visual Wake Words (VWW) dataset, derived from MS-COCO, has served as the de facto standard for TinyML person detection<sup>[26]</sup> <sup>[17]</sup>. However, VWW contains only approximately 60,000 images, limiting its effectiveness for training robust models.

Recent advances include the Wake Vision dataset, which provides over 6 million images—approximately 100× larger than VWW—with a cleaned test set exhibiting only 2.2% label error rate<sup>[26]</sup> <sup>[27]</sup>. Wake Vision addresses critical limitations of previous datasets by

providing comprehensive coverage of diverse scenarios including variations in:

- Perceived gender and age demographics
- Distance from camera (close-range to distant detection)
- Lighting conditions (bright, dim, nighttime)
- Depiction types (photographs, illustrations, artistic representations)

The dataset is released under a permissive CC-BY 4.0 license and is directly accessible through TensorFlow Datasets and Hugging Face, eliminating the regeneration requirements that complicated VWW usage<sup>[26]</sup> <sup>[27]</sup>.

## 4.2 Model Architectures

**MobileNet and MobileNet-SSD:** MobileNet has become the predominant architecture for embedded vision due to its efficient use of depthwise separable convolutions, which dramatically reduce computational cost compared to standard convolutions<sup>[28]</sup> <sup>[29]</sup> <sup>[30]</sup>. The Single Shot MultiBox Detector (SSD) framework combined with MobileNet backbone enables real-time object detection with bounding box localization<sup>[28]</sup> <sup>[29]</sup>.

Research on MobileNet-SSD for embedded systems reports average precision exceeding 99% for vehicle detection and 97-99% for person detection in controlled environments<sup>[28]</sup> <sup>[29]</sup>. However, performance degrades in challenging conditions including variable lighting, occlusions, and distant subjects<sup>[30]</sup>.

**FOMO (Faster Objects, More Objects):** FOMO represents a specialized architecture designed explicitly for resource-constrained devices<sup>[23]</sup> <sup>[24]</sup>. Unlike traditional object detectors that predict bounding boxes, FOMO predicts object center points, reducing computational overhead substantially. Research comparing FOMO and YOLO variants on microcontrollers demonstrates that FOMO can achieve F1 scores between 58-95% while maintaining inference latency below 1 millisecond on some configurations<sup>[23]</sup>.

**TinyYOLO and YOLOv5/v8:** YOLO (You Only Look Once) architectures offer excellent detection performance but require careful optimization for microcontroller deployment<sup>[23]</sup> <sup>[31]</sup> <sup>[24]</sup>. TinyissimoYOLO, an ultra-lightweight YOLO variant, demonstrates near 3 FPS inference on the GAP9 RISC-V microcontroller while achieving state-of-the-art mean object error count for small object detection<sup>[23]</sup>.

Comparative studies on pothole detection using TinyML reveal that FOMO models generally demonstrate superior performance-efficiency trade-offs compared to YOLO variants when deployed on severely resource-constrained platforms<sup>[24]</sup>.

## 4.3 Adaptive Tiling for Small Object Detection

Traditional computer vision models struggle with small object detection, a common scenario in doorway monitoring where individuals may appear at various distances. Adaptive tiling methods divide high-resolution images into smaller overlapping tiles, enabling models to detect objects at multiple scales<sup>[32]</sup> <sup>[23]</sup>.

Research on the DSORT-MCU framework demonstrates that adaptive tiling boosts F1-scores by up to 225% for FOMO networks and up to 89% for TinyissimoYOLO while reducing average object count error significantly<sup>[32]</sup>. The technique enables detection on low-power MCUs with no accuracy compromise compared to large-scale detection models<sup>[32]</sup>.

## 5. Edge Computing and Privacy Preservation

### 5.1 Edge Computing Architecture

Edge computing processes data locally at or near the source of data generation, contrasting with traditional cloud-centric architectures that transmit raw data to remote servers<sup>[2]</sup> <sup>[33]</sup> <sup>[34]</sup>. For surveillance applications, edge computing provides several critical advantages:

**Latency Reduction:** Local processing eliminates round-trip communication delays to cloud servers. Research on ESP32-based edge computing for object recognition demonstrates that edge servers significantly reduce bandwidth usage and latency, effectively alleviating cloud server load<sup>[2]</sup>.

**Privacy Enhancement:** Processing images locally without cloud transmission addresses fundamental privacy concerns inherent in video surveillance systems<sup>[33]</sup> <sup>[35]</sup> <sup>[34]</sup>. A privacy-preserving framework for image classification proposes that autoencoders trained at edge devices generate latent representations that preserve utility while protecting sensitive visual information<sup>[34]</sup>.

**Bandwidth Optimization:** Transmitting inference results (person detected/not detected) rather than raw image data reduces network bandwidth consumption by orders of magnitude<sup>[2]</sup> <sup>[33]</sup>. This becomes particularly important for systems with limited connectivity or metered data plans.

## 5.2 Privacy-Preserving Techniques

Research on privacy-preserving edge computing for face recognition demonstrates that differential privacy mechanisms can provide strong privacy guarantees while maintaining classification accuracy<sup>[36]</sup>. The approach splits deep neural networks between user devices and edge servers, protecting both private data and model parameters with minimal local computational overhead<sup>[36]</sup>.

An intelligent monitoring system with privacy preservation achieves 95.3% accuracy while blocking direct data transmission between camera modules and external servers through an intermediate edge AI module<sup>[35]</sup>. This architecture ensures that personally identifiable visual information never leaves the local processing environment.

## 6. Communication Protocols and WhatsApp Integration

### 6.1 Messaging API Integration

Several approaches enable ESP32-based systems to send notifications via messaging platforms:

**CallMeBot API:** The CallMeBot service provides a straightforward HTTP-based API for sending WhatsApp messages from embedded devices<sup>[3]</sup><sup>[37]</sup><sup>[38]</sup>. The API requires registration of the target phone number and generation of an API key. ESP32 code constructs HTTP POST requests with URL-encoded message content, enabling text and basic alert transmission<sup>[3]</sup>.

Implementation involves configuring Wi-Fi credentials, phone number with country code, and API key. The HTTP client library manages connection establishment and request transmission. Response codes indicate success (200) or failure, enabling error handling and retry logic<sup>[3]</sup><sup>[37]</sup>.

**Limitations and Alternatives:** CallMeBot and similar services impose rate limits and may face reliability issues during high-volume usage. Alternative approaches include:

- **Telegram Bot API:** Offers more robust programmatic access and supports image transmission alongside text messages<sup>[39]</sup><sup>[37]</sup>
- **MQTT Publish-Subscribe:** Enables decoupled communication between IoT devices and notification services through message brokers<sup>[2]</sup><sup>[40]</sup>
- **SMS via GSM Modules:** Provides notification capability independent of internet connectivity, though at higher cost per message<sup>[41]</sup>

### 6.2 Image Transmission Considerations

While text notifications provide basic alerting capability, transmitting captured images substantially enhances the utility of intrusion detection systems by providing visual verification. However, image transmission from ESP32 presents technical challenges:

**Encoding and Compression:** JPEG encoding reduces image file sizes significantly, but ESP32 computational limitations constrain compression quality and speed. Research indicates that transmitting 640×480 JPEG images from ESP32-CAM to cloud storage or messaging services requires careful optimization of image quality parameters to balance file size and visual fidelity<sup>[42]</sup>.

**Base64 Encoding:** Many HTTP-based APIs require image data in base64-encoded format, which increases payload size by approximately 33%. This overhead impacts transmission time and memory requirements on resource-constrained devices.

**Network Stability:** Image transmission requires sustained network connectivity during the entire upload process. Implementations must handle connection interruptions gracefully, potentially implementing chunked transfer or retry mechanisms.

## 7. Real-Time Detection and Response Systems

### 7.1 System Architecture Design

Effective real-time intrusion detection systems integrate multiple components operating in coordinated fashion:

**Sensor Fusion:** Motion detection sensors (PIR) serve as low-power triggers that activate the camera module and inference pipeline only when movement is detected<sup>[39]</sup><sup>[40]</sup>. This event-driven architecture maximizes battery life by avoiding continuous camera operation.

**Processing Pipeline:** The data flow typically follows this sequence:

1. Motion sensor triggers camera activation
2. ESP32 captures image at optimized resolution

3. Image preprocessing (resizing, normalization) prepares data for inference
4. TinyML model executes person detection inference
5. Post-processing interprets model outputs and applies decision thresholds
6. Notification system transmits alerts if person is detected
7. Optional image storage or transmission for verification<sup>[2]</sup> <sup>[17]</sup> <sup>[40]</sup>

**Edge AI Module:** Research on intelligent monitoring systems emphasizes the importance of dedicated edge AI modules that process visual data locally before any external communication occurs<sup>[35]</sup>. This architecture prevents transmission of sensitive visual information while enabling intelligent decision-making.

## 7.2 Performance Metrics

Evaluation of real-time detection systems requires consideration of multiple performance dimensions:

**Detection Accuracy:** The fundamental measure of model effectiveness. Research on VGG16 convolutional neural networks for low-light person detection reports 100% accuracy on test sets, though real-world performance typically ranges 85-95% depending on environmental conditions<sup>[43]</sup>.

**Latency:** Time from event occurrence (person arrival) to notification delivery. Research on IoT-based emergency response systems demonstrates that well-designed architectures achieve end-to-end latencies under 500 milliseconds<sup>[40]</sup>. For doorway monitoring, latency requirements are less stringent, with 2-5 second delays generally acceptable.

**False Positive Rate:** Excessive false alarms reduce user trust and system utility. Person detection models trained on diverse datasets demonstrate false positive rates below 5% in controlled scenarios<sup>[26]</sup>.

**Energy Consumption:** Critical for battery-powered deployments. Research indicates that duty-cycled operation using motion sensor triggers can extend operational lifetime from days to months compared to continuous processing<sup>[6]</sup>.

## 7.3 Environmental Robustness

Real-world deployment environments present challenges not captured in laboratory testing:

**Lighting Variations:** Person detection accuracy degrades substantially in low-light and nighttime conditions. Research on low-light night vision image classification demonstrates that image enhancement algorithms (CLAHE) combined with appropriate neural network architectures (VGG16) can achieve 92% classification accuracy despite challenging lighting<sup>[43]</sup>.

**Occlusions and Partial Views:** Doorway scenarios frequently involve partial body visibility as individuals enter or exit. Training datasets must include examples of partial occlusions to enable robust detection<sup>[26]</sup>.

**Weather Conditions:** Outdoor deployments must contend with rain, fog, and precipitation that degrade image quality. Weatherproofing and lens protection become important hardware considerations.

## 8. Related Systems and Comparative Analysis

### 8.1 Commercial Smart Doorbell Systems

Commercial products such as Ring and Nest Hello provide comprehensive doorway monitoring solutions with cloud-based processing, mobile applications, and video storage capabilities. However, these systems exhibit several limitations addressed by edge-based approaches:

**Subscription Requirements:** Cloud storage and advanced features typically require ongoing subscription fees, increasing total cost of ownership<sup>[44]</sup> <sup>[45]</sup>.

**Privacy Concerns:** Video data transmitted to cloud servers raises privacy concerns, particularly given incidents of unauthorized access and data breaches<sup>[45]</sup>.

**Connectivity Dependence:** Cloud-dependent systems become non-functional during internet outages, eliminating security coverage precisely when it may be most needed<sup>[2]</sup>.

## 8.2 Academic Research Projects

Several research projects have explored intelligent doorway monitoring:

**Face Recognition Door Lock Systems:** Systems combining face recognition with automated door control provide enhanced security through biometric authentication<sup>[41] [44]</sup>. However, face recognition introduces more significant privacy concerns and computational requirements compared to simple person detection.

**IoT-Based Smart Doorbell Systems:** Research prototypes demonstrate feasibility of ESP32 or Raspberry Pi-based systems with camera integration, visitor recognition, and notification capabilities<sup>[44] [45]</sup>. These systems validate the technical approach but often lack optimization for production deployment.

**Privacy-Preserving Surveillance:** Research on privacy-aware monitoring systems demonstrates techniques for detecting human presence and activity patterns without capturing identifiable visual information<sup>[35] [46]</sup>. These approaches employ skeletal tracking, silhouette detection, or encrypted visual features to preserve privacy while maintaining monitoring capability.

## 8.3 Comparative Advantages of EagleEye Approach

The EagleEye system distinguishes itself through several design choices:

**Local Processing:** Complete person detection inference executes on-device, eliminating cloud dependence and protecting privacy<sup>[1] [2]</sup>.

**Cost Effectiveness:** Utilizing affordable commodity hardware (ESP32-CAM modules cost \$10-15) makes the system accessible to broader user populations compared to commercial alternatives requiring \$100-200 initial investment plus subscriptions<sup>[1]</sup>.

**Minimal Latency:** Edge processing eliminates network round-trip delays, enabling sub-second response times<sup>[2] [40]</sup>.

**Independence from Internet:** System can operate with intermittent connectivity, requiring network access only for notification delivery rather than core detection functionality<sup>[1] [2]</sup>.

## 9. Challenges and Limitations

### 9.1 Technical Constraints

**Computational Limitations:** The ESP32 platform imposes strict constraints on model complexity and inference speed. Research consistently reports that ESP32 achieves 1-3 FPS for person detection tasks, which is acceptable for doorway monitoring but insufficient for continuous video surveillance<sup>[18] [20] [21]</sup>.

**Model Size Restrictions:** TinyML models must fit within approximately 400-500 KB of flash memory while utilizing less than 200 KB of RAM during inference<sup>[15] [25]</sup>. These constraints necessitate careful architecture selection and aggressive model compression.

**Accuracy-Efficiency Trade-offs:** Reducing model complexity to meet resource constraints inevitably impacts detection accuracy. Research on TinyML indicates that models are more sensitive to dataset quality than larger models, requiring careful dataset curation<sup>[26]</sup>.

### 9.2 Environmental Challenges

**Variable Lighting:** Person detection accuracy degrades substantially in low-light conditions. While image enhancement algorithms provide some improvement, fundamental limitations exist when insufficient light information is captured<sup>[43]</sup>.

**Weather Interference:** Precipitation, fog, and lens obstruction significantly impact image quality in outdoor deployments. Hardware enclosures with lens protection and heating elements may be necessary.

**Distance and Angle Variations:** Detection performance varies substantially based on subject distance and camera angle. Training datasets must encompass the full range of expected scenarios<sup>[26]</sup>.

### 9.3 System Integration Challenges

**Power Management:** Balancing continuous monitoring capability with battery life requires sophisticated power management strategies including sleep modes, event-driven activation, and energy harvesting<sup>[6] [8]</sup>.

**Network Reliability:** WhatsApp notification delivery depends on network connectivity. Systems must implement message queuing and retry logic to handle temporary connectivity losses<sup>[40]</sup>.

**False Alarm Management:** Minimizing false positives while maintaining high detection sensitivity represents an ongoing challenge. Multi-stage verification or confirmation mechanisms may be necessary.

## 10. Future Directions and Emerging Trends

### 10.1 Hardware Advances

**Specialized Neural Accelerators:** Next-generation microcontrollers increasingly integrate dedicated neural processing units (NPUs) or AI accelerators that dramatically improve inference performance while reducing power consumption<sup>[4] [8]</sup>. Examples include the MAX78000 with integrated CNN accelerator and various Cortex-M processors with vector extensions.

**Neuromorphic Computing:** Brain-inspired neuromorphic processors offer extreme energy efficiency for event-based vision processing, potentially enabling always-on visual monitoring at sub-milliwatt power levels<sup>[8]</sup>.

**Enhanced ESP32 Variants:** ESP32-S3 and upcoming variants feature increased memory, improved processing capabilities, and enhanced camera interfaces that address current platform limitations<sup>[4]</sup>.

### 10.2 Algorithm Improvements

**Self-Supervised Learning:** Emerging techniques enable models to learn effective representations from unlabeled data, dramatically reducing dataset requirements and annotation costs<sup>[47]</sup>. This becomes particularly valuable for domain-specific applications where labeled data is scarce.

**Neural Architecture Search:** Automated techniques for discovering optimal model architectures for specific hardware platforms promise to identify configurations that better balance accuracy and efficiency than manually designed networks<sup>[8]</sup>.

**Continual Learning:** Enabling deployed models to adapt to changing environmental conditions without full retraining enhances long-term system robustness<sup>[46] [8]</sup>.

### 10.3 Application Expansion

**Multi-Modal Sensing:** Integrating audio detection (sound recognition), thermal imaging, and traditional vision creates more robust systems resistant to limitations of individual sensing modalities<sup>[48] [40]</sup>.

**Behavioral Analytics:** Beyond simple presence detection, analyzing patterns of activity over time enables more sophisticated applications including occupancy analytics, behavior anomaly detection, and predictive maintenance<sup>[18] [49]</sup>.

**Federated Learning:** Collaborative model training across multiple deployed devices while preserving privacy represents an emerging approach for improving system performance through distributed learning<sup>[46]</sup>.

## 11. Conclusion

The convergence of TinyML, edge computing, and affordable embedded hardware has created unprecedented opportunities for intelligent, privacy-preserving security systems. The literature demonstrates that ESP32-based person detection with local image processing represents a technically feasible approach that addresses fundamental limitations of cloud-dependent alternatives.

Key findings from the literature include:

**Technical Feasibility:** Research conclusively demonstrates that appropriately optimized neural networks can execute person detection inference on ESP32 hardware at acceptable frame rates (1-3 FPS) and accuracy levels (85-95%)<sup>[2] [15] [26]</sup>.

**Energy Viability:** TinyML approaches achieve dramatic energy efficiency improvements compared to traditional machine learning systems, enabling extended battery-powered operation when combined with intelligent power management<sup>[6] [8]</sup>.

**Privacy Advantages:** Local processing eliminates the need to transmit visual data to external servers, fundamentally addressing privacy concerns inherent in cloud-based surveillance<sup>[2] [35] [34]</sup>.

**Cost Effectiveness:** Utilizing commodity ESP32-CAM hardware (\$10-15) makes intelligent monitoring accessible to broader populations compared to commercial alternatives requiring substantial upfront investment and ongoing subscriptions<sup>[1]</sup>.

**Integration Challenges:** Successful implementation requires careful attention to model optimization, power management, environmental robustness, and system integration. The ESP32 platform imposes meaningful constraints that necessitate trade-offs between capability and resource utilization<sup>[18] [15] [22]</sup>.

The EagleEye Smart Intrusion System represents a practical application of these research findings, demonstrating how TinyML enables intelligent functionality on resource-constrained embedded devices. Future enhancements leveraging specialized hardware accelerators, improved algorithms, and multi-modal sensing promise to further expand the capabilities and robustness of edge-based security systems.

As TinyML research continues its rapid growth trajectory, increasingly sophisticated applications will become feasible on progressively more constrained hardware platforms. This democratization of intelligent functionality has profound implications for security, privacy, accessibility, and the evolution of the IoT ecosystem.

## References

- <sup>[1]</sup> EagleEye Smart Intrusion System Project Documentation
- <sup>[18]</sup> ESP32-CAM Project: Human Detection, Image Capture, Pattern Recognition, and Telegram Alerts. Reddit Discussion, 2024.
- <sup>[42]</sup> Esp32cam send captured photo to WhatsApp. YouTube, 2020.
- <sup>[12]</sup> Simple ESP32-CAM Object Detection. YouTube, 2023.
- <sup>[50]</sup> ESP32-CAM: TinyML Image Classification - Fruits vs Veggies. [Hackster.io](https://hackster.io), 2023.
- <sup>[39]</sup> ESP32 Sending Alert Message To WhatsApp. [TechTONions.com](https://techtonions.com), 2025.
- <sup>[2]</sup> Design and Implementation of ESP32-Based Edge Computing for Object Recognition. PMC, 2025.
- <sup>[33]</sup> Smart Video Surveillance System Based on Edge Computing. PMC, 2021.
- <sup>[32]</sup> Detecting Small Objects in Real-Time on Microcontroller Units. arXiv, 2024.
- <sup>[19]</sup> How to Perform Object Detection Directly on ESP32-CAM? Arduino Forum, 2024.
- <sup>[4]</sup> From Tiny Machine Learning to Tiny Deep Learning: A Survey. arXiv, 2025.
- <sup>[51]</sup> Multicamera edge-computing system for persons indoor location and tracking. ScienceDirect, 2023.
- <sup>[52]</sup> Microcontrollers for computer vision. Reddit Discussion, 2023.
- <sup>[53]</sup> 6 Popular ESP32 AI Applications Using TinyML in 2024. DFRobot, 2025.
- <sup>[15]</sup> Camera Image Processing on ESP32 Microcontroller with Help of Convolutional Neural Network. Semantic Scholar, 2022.
- <sup>[54]</sup> Evaluation of Computer Vision-Based Person Detection on Low-Cost Embedded Systems. PDF, 2023.
- <sup>[23]</sup> Detecting Small Objects in Real-Time on Microcontroller Units. arXiv, 2024.
- <sup>[55]</sup> ESP32 CAM Object Detection & Identification with OpenCV. How2Electronics.
- <sup>[16]</sup> TinyML for Image Processing. ScienceDirect, 2024.
- <sup>[56]</sup> Human Activity Recognition on Embedded Devices: An Edge AI Approach. PDF, 2025.
- <sup>[57]</sup> Electronic component recognition based on microcontroller and machine vision. ACM Digital Library, 2025.
- <sup>[3]</sup> ESP32: Send Messages to WhatsApp. Random Nerd Tutorials, 2024.
- <sup>[28]</sup> Real Time Object Detection Using MobileNet-SSD. PDF, 2024.
- <sup>[20]</sup> Raspberry Pi Pico Person Detection with TinyML TensorFlow Lite. YouTube, 2021.
- <sup>[37]</sup> How to send messages to WhatsApp or Telegram from ESP32. Engineers Garage, 2024.



- [29] Real-time object detection and recognition using MobileNet-SSD with OpenCV. PDF.
- [11] TensorFlow Lite Micro. Fritz AI, 2023.
- [38] Send a WhatsApp message using ESP32. ElectronicWings, 2024.
- [30] An Improved Object Detection Model for Embedded Systems. IEEE, 2020.
- [31] Open model for person detection? Reddit Discussion, 2024.
- [58] Using WhatsApp with your ESP32. YouTube, 2023.
- [59] Mobilenet-SSDv2: An Improved Object Detection Model for Embedded Systems. PDF.
- [5] A review on TinyML: State-of-the-art and prospects. ScienceDirect.
- [41] Known and Unknown Face Smart Home Door Lock System using AI and Edge Computing. PDF.
- [43] Research on Target Image Classification in Low-Light Night Vision. PMC, 2024.
- [60] Real-Time Edge AI: Embedded Vision for Smarter Systems. Logic Fruit, 2025.
- [35] Intelligent Monitoring System with Privacy Preservation Based on Edge AI. PMC, 2023.
- [61] Modular Neural Networks for Low-Power Image Classification on Embedded Devices. ACM, 2020.
- [49] Real-Time Human Detection and Counting System Using Deep Learning Computer Vision Techniques, 2022.
- [48] How Edge computing in surveillance Improves Security. Pioneer Security, 2025.
- [62] Low-power scalable multilayer optoelectronic neural networks. Nature, 2024.
- [63] Real-Time Human Action Recognition on Embedded Platforms. arXiv, 2024.
- [64] Edge AI: A survey. ScienceDirect.
- [65] Image recognition based on lightweight convolutional neural network. ScienceDirect.
- [66] An optimized lightweight real-time detection network model for IoT embedded devices. Nature, 2025.
- [67] Improving Object Detection Accuracy with ESP32-CAM and OpenCV. Sparkfun Community, 2025.
- [44] Smart Doorbell Security System Using IoT, 2021.
- [22] ESP32-CAM Troubleshooting Guide: Most Common Problems Fixed. Random Nerd Tutorials, 2024.
- [24] TinyML-Based Pothole Detection: A Comparative Analysis of YOLO and FOMO. Semantic Scholar.
- [68] A Supervised Intrusion Detection System for Smart Home IoT Devices. PDF.
- [69] ESP32 CAM with Python OpenCV Yolo V3 for object detection. YouTube, 2023.
- [21] Person detection model on ESP32-WROVER? Arduino Forum, 2022.
- [70] Tiny Machine Learning and On-Device Inference: A Survey. PMC, 2025.
- [46] Enhancing Smart Home Security: Anomaly Detection and Face Recognition. PMC, 2023.
- [71] Simple ESP32-CAM Object Detection. DroneBot Workshop Forums, 2023.
- [72] Tiny machine learning empowers climbing inspection robots. ScienceDirect.
- [73] Recent endeavors in machine learning-powered intrusion detection systems. ScienceDirect.
- [34] Efficient Privacy Preserving Edge Computing Framework for Image Classification. arXiv, 2020.
- [6] Evaluation of the energy viability of smart IoT sensors using TinyML for computer vision applications. MedCrave, 2023.
- [36] A Privacy-Preserving Deep Learning Approach for Face Recognition with Edge Computing. PDF.

- [7] A Comprehensive Survey on Tiny Machine Learning for Human Activity Recognition. IEEE, 2025.
- [74] Enhancing the Energy Efficiency and Robustness of tinyML Computer Vision. ACM, 2022.
- [75] Privacy-preserving geo-tagged image search in edge–cloud computing. ScienceDirect.
- [9] Optimizing TinyML: The Impact of Reduced Data Acquisition Rates. arXiv, 2024.
- [76] Emerging trends and strategic opportunities in tiny machine learning. ScienceDirect.
- [77] Privacy-preserving Multi-source Image Retrieval in Edge Computing. IEEE, 2022.
- [10] Combinative model compression approach for enhancing 1D CNN performance. ScienceDirect.
- [78] Benchmarking Energy and Latency in TinyML. arXiv, 2025.
- [26] Wake Vision: A Tailored Dataset and Benchmark Suite for TinyML Person Detection. arXiv, 2024.
- [13] How to Deploy Model to Arduino? TinyWorld, 2021.
- [45] Usage and impact of the internet-of-things-based smart home technology. PMC, 2022.
- [79] TinyML person detection. The TinyML Book, 2025.
- [14] Deployment - Edge Impulse Documentation, 2025.
- [80] A Systematic Study of Data Security Issues in Smart Home IOT Devices. PDF.
- [17] Embedded Machine Learning for Person Detection. [Hackster.io](https://hackster.io), 2022.
- [81] Edge Impulse with the Nano 33 BLE Sense. Arduino Documentation, 2023.
- [82] A review of Internet of Things for Smart Homes Environment. PDF.
- [27] Introducing Wake Vision: A High-Quality, Large-Scale Dataset for Person Detection in TinyML. TensorFlow Blog, 2024.
- [83] How to Deploy a Trained Model to Arduino. YouTube, 2022.
- [84] A critical literature review of security and privacy in smart home healthcare systems. ScienceDirect.
- [40] Developing real-time IoT-based public safety alert and emergency response system. PMC, 2025.
- [25] TinyML-Based In-Pipe Feature Detection for Miniature Robots. PMC, 2025.
- [8] TinyML: Why the Future of Machine Learning is Tiny and Bright. SIGARCH, 2024.
- [85] Evaluation of Embedded Devices for Real-Time Video Lane Detection. IEEE, 2022.
- [47] Key Trends in Computer Vision for 2025. [ImageVision.ai](https://imagevision.ai), 2024.
- [86] Embedded Real-time System Trusted Protection Technology. ACM, 2023.
- [87] Software Engineering Approaches for TinyML Based IoT Embedded Systems. PDF.
- [88] Best Way to Implement Real-Time Notifications. Reddit Discussion, 2025.
- [89] 2025 Computer Vision Trends: The Future of AI Unveiled. Viso Suite, 2024.

✳

1. EagleEye\_Smart\_Intrusion\_System-1.pptx
2. <https://pmc.ncbi.nlm.nih.gov/articles/PMC11945263/>
3. <https://randomnerdtutorials.com/esp32-send-messages-whatsapp/>
4. <https://arxiv.org/html/2506.18927v1>
5. <https://www.sciencedirect.com/science/article/pii/S1319157821003335>
6. <https://medcraveonline.com/IRATJ/evaluation-of-the-energy-viability-of-smart-iot-sensors-using-tinyml-for-computer-vision-applications-a-case-study.html>

7. <https://ieeexplore.ieee.org/iel8/6488907/11121332/10979983.pdf>
8. <https://www.sigarch.org/tiny-machine-learning-the-future-of-ml-is-tiny-and-bright/>
9. <https://arxiv.org/html/2409.10942v1>
10. <https://www.sciencedirect.com/science/article/pii/S2542660524003445>
11. <https://fritz.ai/what-is-tensorflow-lite-micro/>
12. [https://www.youtube.com/watch?v=HDRVZ\\_BYd08](https://www.youtube.com/watch?v=HDRVZ_BYd08)
13. <https://pelinbalci.com/tinyml/2021/11/28/Deploy-To-Arduino.html>
14. <https://docs.edgeimpulse.com/studio/projects/deployment>
15. <https://www.semanticscholar.org/paper/Camera-Image-Processing-on-ESP32-Microcontroller-of-Sineglazov-Khotsyanovsky/a63f79314f667b27cb4407dcf2f5ad9363568b59>
16. <https://www.sciencedirect.com/science/article/pii/S1877050924027285/pdf?md5=ed4eb3e94c3d4a4381f5d5de172c8b4e&pid=1-s2.0-S1877050924027285-main.pdf>
17. <https://www.hackster.io/anaferraz/embedded-machine-learning-for-person-detection-664030>
18. [https://www.reddit.com/r/esp32/comments/1gxr3iy/esp32cam\\_project\\_human\\_detection\\_image\\_capture/](https://www.reddit.com/r/esp32/comments/1gxr3iy/esp32cam_project_human_detection_image_capture/)
19. <https://forum.arduino.cc/t/how-to-perform-object-detection-directly-on-esp32-cam/1331647>
20. <https://www.youtube.com/watch?v=yK1eu9uQaBs>
21. <https://forum.arduino.cc/t/person-detection-model-on-esp32-wrover/1001073>
22. <https://randomnerdtutorials.com/esp32-cam-troubleshooting-guide/>
23. <https://arxiv.org/html/2410.16769v1>
24. <https://www.semanticscholar.org/paper/TinyML-Based-Pothole-Detection:-A-Comparative-of-Silva-Flores/ab5241c731aa01c23266aac065d90ce866fda9e6>
25. <https://pmc.ncbi.nlm.nih.gov/articles/PMC11945833/>
26. <https://arxiv.org/html/2405.00892v3>
27. <https://blog.tensorflow.org/2024/12/introducing-wake-vision-new-dataset-for-person-detection-in-tinyml.html>
28. [https://www.ijesat.com/ijesat/files/V24I0534\\_1715094414.pdf](https://www.ijesat.com/ijesat/files/V24I0534_1715094414.pdf)
29. <https://www.ijert.org/research/real-time-object-detection-and-recognition-using-mobilenet-ssd-with-opencv-IJERTV11IS010070.pdf>
30. <https://ieeexplore.ieee.org/document/9219319/>
31. [https://www.reddit.com/r/MachineLearning/comments/1b73br3/d\\_open\\_model\\_for\\_person\\_detection/](https://www.reddit.com/r/MachineLearning/comments/1b73br3/d_open_model_for_person_detection/)
32. <https://arxiv.org/abs/2410.16769>
33. <https://pmc.ncbi.nlm.nih.gov/articles/PMC8122948/>
34. <https://arxiv.org/abs/2005.04563>
35. <https://pmc.ncbi.nlm.nih.gov/articles/PMC10536670/>
36. <https://www.usenix.org/system/files/conference/hotedge18/hotedge18-papers-mao.pdf>
37. <https://www.engineersgarage.com/send-messages-to-whatsapp-telegram-from-esp32/>
38. <https://www.electronicwings.com/esp32/send-a-whatsapp-message-using-esp32>
39. <https://techtonions.com/esp32-sending-alert-message-to-whatsapp/>
40. <https://pmc.ncbi.nlm.nih.gov/articles/PMC12334619/>
41. <https://www.jetir.org/papers/JETIR2205679.pdf>
42. <https://www.youtube.com/watch?v=VaF13VW6zTo>
43. <https://pmc.ncbi.nlm.nih.gov/articles/PMC11507526/>
44. <https://ijrst.com/IJRST2182111>
45. <https://pmc.ncbi.nlm.nih.gov/articles/PMC9667851/>
46. <https://pmc.ncbi.nlm.nih.gov/articles/PMC10422449/>
47. <https://imagevision.ai/blog/trends-in-computer-vision-from-2024-breakthroughs-to-2025-blueprints/>
48. <https://www.pioneersecurity.com/edge-computing-in-surveillance/>
49. <https://ojs.bonviewpress.com/index.php/AIA/article/view/391>
50. <https://www.hackster.io/mjrobot/esp32-cam-tinyml-image-classification-fruits-vs-veggies-4ab970>
51. <https://www.sciencedirect.com/science/article/pii/S2542660523002639>
52. [https://www.reddit.com/r/microcontrollers/comments/17avd7d/microcontrollers\\_for\\_computer\\_vision/](https://www.reddit.com/r/microcontrollers/comments/17avd7d/microcontrollers_for_computer_vision/)
53. <https://www.dfrobot.com/blog-13902.html>

54. <https://www.dei.unipd.it/~nbello/papers/Pasti2023.pdf>
55. <https://how2electronics.com/esp32-cam-based-object-detection-identification-with-opencv/>
56. <https://www.scitepress.org/Papers/2025/134753/134753.pdf>
57. <https://dl.acm.org/doi/10.1145/3757940.3757978>
58. <https://www.youtube.com/watch?v=Tp8CTeV5j1Q>
59. <https://engineersplanet.com/wp-content/uploads/2024/01/Mobilenet-An-Improved-Object-Detection-Model-for-Embedded-Systems.pdf>
60. <https://www.logic-fruit.com/blog/embedded/real-time-embedded-ai/>
61. <https://dl.acm.org/doi/10.1145/3408062>
62. <https://www.nature.com/articles/s41467-024-55139-4>
63. <https://arxiv.org/html/2409.05662v2>
64. <https://www.sciencedirect.com/science/article/pii/S2667345223000196>
65. <https://www.sciencedirect.com/science/article/pii/S0262885624001410>
66. <https://www.nature.com/articles/s41598-025-88439-w>
67. <https://community.sparkfun.com/t/improving-object-detection-accuracy-with-esp32-cam-and-opencv/64614>
68. [https://orca.cardiff.ac.uk/id/eprint/123767/1/A Supervised Intrusion Detection System for SmartHome IoT Devices.pdf](https://orca.cardiff.ac.uk/id/eprint/123767/1/A%20Supervised%20Intrusion%20Detection%20System%20for%20SmartHome%20IoT%20Devices.pdf)
69. <https://www.youtube.com/watch?v=npJsmbFZiMg>
70. <https://pmc.ncbi.nlm.nih.gov/articles/PMC12115890/>
71. <https://forum.dronebotworkshop.com/2023-videos/simple-esp32-cam-object-detection/>
72. <https://www.sciencedirect.com/science/article/abs/pii/S0952197624007760>
73. <https://www.sciencedirect.com/science/article/abs/pii/S1084804524001024>
74. <https://dl.acm.org/doi/10.1145/3591466>
75. <https://www.sciencedirect.com/science/article/abs/pii/S221421262400111X>
76. <https://www.sciencedirect.com/science/article/pii/S0925231225014183>
77. <https://ieeexplore.ieee.org/document/9964114/>
78. <https://arxiv.org/html/2505.15622v1>
79. <https://www.thetinymlbook.com/resources/tinyml-person-detection>
80. <https://digitalcommons.library.uab.edu/cgi/viewcontent.cgi?article=1574&context=etd-collection>
81. <https://docs.arduino.cc/tutorials/nano-33-ble-sense/edge-impulse>
82. <http://ijeais.org/wp-content/uploads/2021/11/IJAMR211101.pdf>
83. <https://www.youtube.com/watch?v=uUh61R8Hu0g>
84. <https://www.sciencedirect.com/science/article/pii/S2096720923000532>
85. <https://ieeexplore.ieee.org/document/9838167/>
86. <https://dl.acm.org/doi/10.1145/3586102.3586112>
87. [https://scholarworks.boisestate.edu/cgi/viewcontent.cgi?article=1354&context=cs\\_facpubs](https://scholarworks.boisestate.edu/cgi/viewcontent.cgi?article=1354&context=cs_facpubs)
88. [https://www.reddit.com/r/FlutterDev/comments/1ire2q5/best way to implement realtime notifications fcm/](https://www.reddit.com/r/FlutterDev/comments/1ire2q5/best_way_to_implement_realtime_notifications_fcm/)
89. <https://viso.ai/computer-vision/computer-vision-trends/>